Experiment No: 1

Aim: Print different patterns of asterisk (*) using loops (e.g. triangle of *).

Theory:
The use of loops (for, while) to print patterns in Java. By adjusting the number of iterations, different shapes, like triangles or pyramids, can be generated using asterisks (*). This helps understand nested loops, control flow, and iteration.

Coding:
```
class Pattern3 {
        public static void main(String[] args) {
      int n = 5;
       for (int i = 1; i <= n; i++) {
          for (int j = i; j < n; j++) {
              System.out.print(" ");
          }
          for (int j = 1; j <= (2 * i - 1); j++) {
              System.out.print("*");
          }
          System.out.println();
          }
          }
}
```

Input:
int n = 5;

Output:
```
    *
   ***
  *****
 *******
*********
```

Example :
```
*
**
***
****
*****
```

Application:

This is a common introductory exercise for understanding loops, and can be applied in areas like visual designs, ASCII art, or graphical pattern generation. It's also useful for understanding the basic structure of loops before moving on to more complex algorithms.

Conclusion:

This activity helps reinforce the concept of loops, especially nested loops, and demonstrates how to control the flow of printing patterns in a structured format. Students develop a deeper understanding of how loop iteration can be used for pattern formation.

Web/virtual links:

https://stackoverflow.com/questions/33247702/printing-a-triangle-of-asterisks-using-for-loops-in-java

References:

- *Head First Java* by Kathy Sierra and Bert Bates.
- *Java: The Complete Reference* by Herbert Schildt.

Experiment No: 2

Aim: Build a class Employee which contains details about the employee and compile and run its instance**.**

Theory:
This task involves creating a class Employee to represent an employee in a system. The class should contain attributes like name, employee ID, position, and salary. By creating methods such as constructors, getters, and setters, you can initialize the employee's details and access or modify them. The task emphasizes object-oriented programming principles like encapsulation and class instantiation.

Coding:

```
// Employee.java
class Employee {
  private int id;
   private String name;
   private String designation;
   private double salary;
        // Constructor
   Employee(int id, String name, String designation, double salary) {
      this.id = id;
      this.name = name;
      this.designation = designation;
      this.salary = salary;
        }
        // Method to display employee details
   void display() {
      System.out.println("Employee Details:");
      System.out.println("ID: " + id);
      System.out.println("Name: " + name);
      System.out.println("Designation: " + designation);
      System.out.println("Salary: ₹" + salary);
        }
        // Main method to create and run an instance
        public static void main(String[] args) {
        // Creating an Employee object
      Employee e = new Employee(101, "Ram Jadhav", "Software Engineer", 60000);
        // Displaying employee details
      e.display();
        }
}
```

```
Employee e = new Employee(101, "Ram Jadhav", "Software Engineer", 60000);
```

Output:
Employee Details:
ID: 101
Name: Ram Jadhav
Designation: Software Engineer
Salary: ₹60000.0

Example & Application:
The **Employee** class can be used in various real-world scenarios, such as:

- ✓ **HR Management Systems** – Storing and managing employee records.
- ✓ **Payroll Systems** – Calculating salaries and other employee benefits.
- ✓ **Database Applications** – Handling employee details in enterprise software.

Conclusion:
The **Employee** class demonstrates **Encapsulation** by keeping data private and providing methods to access it.
It allows **structured data management** and helps in building **scalable applications**.
This concept is widely used in **enterprise-level software applications** and **HRM (Human Resource Management) systems**.

Web/virtual links:
- Encapsulation in Java - GeeksforGeeks
- Java Classes and Objects - W3Schools
- Oracle Docs - Java Object-Oriented Programming

References:

- Herbert Schildt, **"Java: The Complete Reference"**, McGraw-Hill Education.
- Cay S. Horstmann, **"Core Java Volume I – Fundamentals"**, Pearson Education.
- Oracle Java Documentation – https://docs.oracle.com/en/java/

Experiment No: 3

Aim: Write a Java program to create a Book class with attributes title, author, and price. Define a parameterized constructor to initialize these attributes. Create two objects of the class and display their details.

Theory:
In Java, a class is a blueprint for creating objects. It contains fields (variables) and methods to define the behavior of objects. A parameterized constructor is a constructor that takes arguments to initialize an object's attributes when it is created.
In this program, we define a Book class with attributes title, author, and price. A parameterized constructor initializes these attributes, and a method displays the book details. We create two objects of the class and display their details.

Coding:
// Book.java

```
class Book {
        private String title;
        private String author;
        private double price;

        // Parameterized Constructor
        Book(String t, String a, double p) {
        title = t;
        author = a;
        price = p;
        }

        // Method to display book details
        void display() {
        System.out.println("Book Details:");
        System.out.println("Title  : " + title);
        System.out.println("Author : " + author);
        System.out.println("Price  : ₹" + price);
      System.out.println(" --------------------------");
        }
        public static void main(String[] args) {
        // Creating two book objects
        Book b1 = new Book("Let Us Java", " Yashavant Kanetkar ", 399.00);
        Book b2 = new Book("The Alchemist", "Paulo Coelho", 299.50);
```

```
// Displaying book details
        b1.display();
        b2.display();
        }
}
```

Input:
Book b1 = new Book("Let Us Java", "Yashavant Kanetkar", 399.00);
Book b2 = new Book("The Alchemist", "Paulo Coelho", 299.50);

Output:
Book Details:
Title : Let Us Java
Author : Yashavant Kanetkar
Price : ₹399.00
- - - - - - - - - - - - - - - - -
Book Details:
Title : The Alchemist
Author : Paulo Coelho
Price : ₹299.5
- - - - - - - - - - - - - - - - -

Example & Application:
Consider a library management system where books need to be stored with their details such as title, author, and price. Instead of setting attributes individually for each book object, a **parameterized constructor** can be used to initialize these values at the time of object creation.
A **real-world application** of this concept is in online bookstores such as Amazon or Flipkart. When a new book is added to the catalog, its details like title, author, and price are stored using a constructor. This ensures that every book object contains complete information, making it easy to manage and display the book collection.

Conclusion :
This program demonstrates how to define a class with attributes, use a parameterized constructor to initialize objects, and display object details. It helps in understanding object-oriented programming concepts like encapsulation and constructors in Java.

Web/virtual links:
- Java Constructors

References:
1. Herbert Schildt, "Java: The Complete Reference," McGraw-Hill Education.
2. Oracle Java Documentation - https://docs.oracle.com/en/java/

Experiment No: 4

Aim: Create a class Employee and encapsulate the data members.

Theory:
In Java, a class is a blueprint for creating objects. It contains fields (variables) and methods to define the behavior of objects. A parameterized constructor is a constructor that takes arguments to initialize an object's attributes when it is created.
A **class** defines the structure and behavior of an object, while an **object** is an instance of a class with actual values assigned to its attributes. The **constructor** helps in initializing these attributes at the time of object creation. The this keyword is used to refer to the current object's attributes when there is ambiguity between instance variables and parameters.
Encapsulation is one of the fundamental principles of Object-Oriented Programming (OOP). It restricts direct access to an object's data and allows modifications through getter and setter methods, ensuring data integrity and security.

Coding:
//Employee.java
```
class Employee {
        // Private data members (Encapsulation)
        private int empId;
        private String name;
        private double salary;

        // Constructor
        Employee(int empId, String name, double salary) {
        this.empId = empId;
        this.name = name;
        this.salary = salary;
        }
        // Getter and Setter methods
        public int getEmpId() {
        return empId;
        }
        public void setEmpId(int empId) {
        this.empId = empId;
        }
        public String getName() {
        return name;
        }
        public void setName(String name) {
```

```
this.name = name;
        }
        public double getSalary() {
        return salary;
        }

        public void setSalary(double salary) {
        this.salary = salary;
        }
        // Display employee details
        void display () {
        System.out.println("Employee ID: " + empId);
        System.out.println("Name: " + name);
        System.out.println("Salary: " + salary);
        }

        public static void main(String[] args) {
        // Creating an Employee object
        Employee emp = new Employee(101, "John Doe", 50000);

        // Displaying employee details
        emp.display ();

        // Modifying employee details using setters
        emp.setSalary(55000);
        System.out.println("Updated Salary: " + emp.getSalary());
        }
}
```

Input:
Employee emp = new Employee(101, "John Doe", 50000);

Output:
Employee ID: 101
Name: John Doe
Salary: 50000.0
Updated Salary: 55000.0

Example & Application:

Consider an employee management system where employee details such as name, ID, and salary need to be stored securely. Using encapsulation, we can restrict direct access to these attributes and provide controlled access through getter and setter methods.

A **real-world application** of this concept is in HR management systems, where employee details are stored securely. Encapsulation prevents unauthorized access to sensitive information such as salary while allowing controlled updates.

Conclusion :

This program demonstrates how to define a class with attributes, use encapsulation through getter and setter methods, and initialize objects. It helps in understanding object-oriented programming concepts like data hiding, security, and constructors in Java.

Web/virtual links:

- Java Encapsulation
- Object-Oriented Programming in Java

References:

1. Herbert Schildt, "Java: The Complete Reference," McGraw-Hill Education.
2. Oracle Java Documentation - https://docs.oracle.com/en/java/

Experiment No: 5

Aim: Print default values of static & instance variables for different data types.

Theory:
Default values for different data types:

- Integer (int, byte, short, long) → 0
- Floating-point (float, double) → 0.0
- Character (char) → '\u0000' (null character)
- Boolean (boolean) → false
- Object references (String, arrays, etc.) → null

Static variables belong to the class rather than any specific instance, and instance variables are unique to each object. Java provides default values to uninitialized variables depending on their data type.

Coding:

```
class DefaultValues {
        // Instance variables
        int a;
        double b;
   boolean c;
        char d;
        String e;

        // Static variables
        static int sa;
        static double sb;
        static boolean sc;
        static char sd;
        static String se;

        // Method to display default values of instance variables
        void displayInstanceVal() {
      System.out.println("Instance Variables:");
      System.out.println("int: " + a);
      System.out.println("double: " + b);
      System.out.println("boolean: " + c);
      System.out.println("char: " + d);
      System.out.println("String: " + e);
        }
```

```
// Method to display default values of static variables
        static void displayStaticVal() {
    System.out.println("Static Variables:");
    System.out.println("int: " + sa);
    System.out.println("double: " + sb);
    System.out.println("boolean: " + sc);
    System.out.println("char: " + sd);
    System.out.println("String: " + se);
        }

        public static void main(String[] args) {
    DefaultValues obj = new DefaultValues();

        // Display default values of instance variables
    obj.displayInstanceVal();

        // Display default values of static variables
    displayStaticVal();
        }
}
```

Input:

```
int a;      // Default: 0
double b;   // Default: 0.0
boolean c;  // Default: false
char d;     // Default: '\u0000' (null character)
String e;   // Default: null
```

Output:

```
Instance Variables:
int: 0
double: 0.0
boolean: false
char:
String: null
Static Variables:
int: 0
double: 0.0
boolean: false
char:
String: null
```

In Java, instance and static variables are assigned default values if not explicitly initialized. This is useful in scenarios where object attributes need to have predefined default values before assignment.
A **real-world application** of default values is in database applications where fields may be uninitialized upon object creation. Java ensures that object properties are assigned meaningful default values, reducing the risk of unexpected behavior.

Conclusion :
This program demonstrates how Java assigns default values to uninitialized instance and static variables. It helps in understanding data type defaults and their significance in object-oriented programming.

Web/virtual links:
- Default Values of Variables in Java – GeeksforGeeks
- Java Variable Types – W3Schools
- Java Static and Instance Variables – JavaTpoint
- Oracle Docs – Java Data Types

References:
1. Herbert Schildt, "Java: The Complete Reference," McGraw-Hill Education.
2. Oracle Java Documentation - https://docs.oracle.com/en/java/

Experiment No: 6

Aim: Create sample classes to understand boxing & unboxing**.**

Theory:
Boxing and Unboxing in Java allow conversion between primitive data types and their corresponding wrapper class objects.
Boxing is the process of converting a primitive type (e.g., int, double) into an object of its corresponding wrapper class (Integer, Double). This allows primitive values to be stored in data structures that only accept objects, such as collections (ArrayList, HashMap).
Unboxing is the reverse process, where an object of a wrapper class is converted back into a primitive type. Java automatically handles these conversions, making it easier to work with objects and primitives interchangeably.
Before Java 5, boxing and unboxing required explicit conversion using wrapper class methods (Integer.valueOf(int), intValue()). From Java 5 onwards, **Autoboxing** and **Auto-unboxing** allow Java to perform these conversions automatically.

Coding:
//Primitive to Wrapper (int to Integer) boxing

```
class WrapperExample1{
	public static void main(String args[]){
		int a=20;

		//converting int to Integer
		Integer i=Integer.valueOf(a);
	//autoboxing compiler will convert Integer.valueOf(a) internally
		Integer j=a;
		System.out.println(a+" "+i+" "+j);
	}
}
```

//Wrapper to Primitive (Integer to int) unboxing

```
class WrapperExample2{
	public static void main(String args[]){
		Integer a=new Integer(3);
		//converting Integer to int
		int i=a.intValue();
		//unboxing, compiler will convert a.intValue() internally
		int j=a;
		System.out.println(a+" "+i+" "+j);
```

Output:
20 20 20
3 3 3

Example & Application:
Boxing and Unboxing are widely used in Java collections, mathematical calculations, and frameworks that require objects instead of primitives.
Boxing and Unboxing are frequently used in **Java Collections** because collections like ArrayList store objects rather than primitive types.

Conclusion :
Boxing and Unboxing simplify working with collections, databases, and APIs that require objects. With **Autoboxing and Auto-unboxing**, Java provides seamless conversion between primitive data types and their wrapper classes, enhancing readability and reducing manual conversions.
Web/virtual links:
https://docs.oracle.com/javase/tutorial/java/data/autoboxing.html
References:
1. Herbert Schildt, "Java: The Complete Reference," McGraw-Hill Education.
2. Oracle Java Documentation - https://docs.oracle.com/en/java/

Experiment No: 07

Aim: Create demo applications to illustrate different types of inheritance.

Theory:
Inheritance is a mechanism where one class acquires the properties and behaviors of another class.
Java supports the following types of inheritance:
1. Single Inheritance: A class inherits from a single parent class.
2. Multilevel Inheritance: A class is derived from another derived class.
3. Hierarchical Inheritance: Multiple classes inherit from the same base class.

Coding:
```
// Parent class representing a general Vehicle
class Vehicle {
	String brand;
	int speed;

	public Vehicle(String brand, int speed) {
	this.brand = brand;
	this.speed = speed;
	}

	public void showDetails() {
	System.out.println("Brand: " + brand + ", Speed: " + speed + " km/h");
	}
}

// Single Inheritance: Car inherits from Vehicle
class Car extends Vehicle {
	int doors;

	public Car(String brand, int speed, int doors) {
	super(brand, speed);
	this.doors = doors;
	}

	public void showCarDetails() {
	showDetails();
	System.out.println("Doors: " + doors);
	}
}
```

```
// Multilevel Inheritance: ElectricCar inherits from Car
class ElectricCar extends Car {
	int batteryCapacity;

	public ElectricCar(String brand, int speed, int doors, int batteryCapacity) {
	super(brand, speed, doors);
	this.batteryCapacity = batteryCapacity;
	}

	public void showElectricCarDetails() {
	showCarDetails();
	System.out.println("Battery Capacity: " + batteryCapacity + " kWh");
	}
}

// Hierarchical Inheritance: Bike also inherits from Vehicle
class Bike extends Vehicle {
	String type;

	public Bike(String brand, int speed, String type) {
	super(brand, speed);
	this.type = type;
	}

	public void showBikeDetails() {
	showDetails();
	System.out.println("Type: " + type);
	}
}

// Main class to test the program
public class InheritanceDemo {
	public static void main(String[] args) {
	System.out.println("--- Single Inheritance ---");
	Car car = new Car("Toyota", 180, 4);
	car.showCarDetails();

	System.out.println("\n--- Multilevel Inheritance ---");
	ElectricCar eCar = new ElectricCar("Tesla", 200, 4, 100);
	eCar.showElectricCarDetails();


	System.out.println("\n--- Hierarchical Inheritance ---");


Bike bike = new Bike("Yamaha", 120, "Sport");
	bike.showBikeDetails();
```

```
        }
}
```

Input:

```
--- Single Inheritance ---
Brand:
Speed:
Doors:

--- Multilevel Inheritance ---
Brand:
Speed:
Doors:
Battery Capacity:

--- Hierarchical Inheritance ---
Brand:
Speed:
Type:
```

Output:

```
--- Single Inheritance ---
Brand: Toyota, Speed: 180 km/h
Doors: 4

--- Multilevel Inheritance ---
Brand: Tesla, Speed: 200 km/h
Doors: 4
Battery Capacity: 100 kWh

--- Hierarchical Inheritance ---
Brand: Yamaha, Speed: 120 km/h
Type: Sport
```

Example & Application:

This program models a real-world scenario where:

- Vehicle is a generic base class.
- Car extends Vehicle (Single Inheritance).
- ElectricCar extends Car (Multilevel Inheritance).
- Bike extends Vehicle, showing Hierarchical Inheritance.

Applications:

- Used in Automobile Industry for structuring vehicle properties.
- Helps in designing banking applications (e.g., Account -> SavingsAccount, CurrentAccount).
- Utilized in e-commerce platforms to manage different product types.

Conclusion:

This Java program successfully demonstrates Single, Multilevel, and Hierarchical Inheritance, showcasing code reuse and hierarchy management.
Web/virtual links: https://docs.oracle.com/en/java/

References:

- *Java: The Complete Reference* by Herbert Schildt

Experiment No: 08

Aim: To implement multilevel inheritance in Java using different packages

Theory:

Inheritance is an important feature of Object-Oriented Programming (OOP) that allows a class to acquire the properties and behaviors of another class. Multilevel inheritance occurs when a class is derived from another derived class, forming a hierarchy of classes.

In Java, we can organize classes into packages to maintain modularity and reusability. A package is a namespace that groups related classes and interfaces.

Example Scenario: We implement an E-commerce User Hierarchy where:

- User (Base Class) defines generic user attributes.
- Customer (Derived Class) extends User and adds customer-specific attributes.
- PrimeCustomer (Derived Class) extends Customer to add premium membership features.

Coding:

1. Package: users (Base Class: User) package users;

```
public class User {
protected String name;
 protected String email;

public User(String name, String email)
 { this.name = name;
this.email = email;
}

public void showDetails()
{ System.out.println("Name: " + name); System.out.println("Email: " + email);
}
}
```

2. Package: customers (Intermediate Class: Customer extends User) package customers;

```
import users.User;

public class Customer extends User
{ protected String address;

public Customer(String name, String email, String address)
{
```

```
super(name, email);
this.address = address;
}

public void showCustomerDetails()
{ showDetails(); System.out.println("Address: " + address);
}
 }
```

Package: primecustomers (Final Derived Class: PrimeCustomer extends Customer) package primecustomers;

```
import customers.Customer;
public class PrimeCustomer extends Customer { private String membershipType;

public PrimeCustomer(String name, String email, String address, String membershipType)
{
super(name, email, address); this.membershipType = membershipType;
}
public void showPrimeCustomerDetails() { showCustomerDetails();
System.out.println("Membership Type: " + membershipType);
}}
```

Main Class to Test Multilevel Inheritance import primecustomers.PrimeCustomer;

```
public class MultilevelInheritanceTest { public static void main(String[] args) {
PrimeCustomer primeUser = new PrimeCustomer("Alice", "alice@example.com", "123 Street, NY", "Gold");
primeUser.showPrimeCustomerDetails();
} }
```

Input:
does not require user input.
Output:
Name: Alice
Email: alice@example.com Address: 123 Street, NY

Example & Application:

- Used in **e-commerce platforms** to categorize users (User -> Customer -> PrimeCustomer).
- Helps in designing **banking systems**, where Account -> SavingsAccount -> PremiumSavingsAccount hierarchy is used.
- Applied in **online learning platforms** to differentiate students (User -> Student -> PremiumStudent).

Conclusion:

- Multilevel inheritance allows building a structured class hierarchy.
- Packages help in organizing code efficiently.
- Promotes code reusability and modularity in Java applications.

Web/virtual links: **https://docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html**
https://docs.oracle.com/javase/tutorial/java/concepts/inheritance.html

References:

- *Effective Java by Joshua Bloch*
- *Head First Java by Kathy Sierra & Bert Bates*

Experiment No: 09

Aim: Create an Array of Employee class and initialize array elements with different employee objects.

Theory:

- In Java, an array is a data structure used to store multiple values of the same type.
- A class is a blueprint for creating objects that encapsulate data and behaviors.
- An array of objects is used when we need to store multiple objects of a class together.
- In this example, the Employee class will have attributes like id, name, designation, and salary, and we will store multiple Employee objects in an array.

Coding:

```
import java.util.Scanner;

// Employee class definition
class Employee {
	int id;
	String name;
	String designation;
	double salary;

	// Constructor to initialize employee details
	public Employee(int id, String name, String designation, double salary) {
	this.id = id;
	this.name = name;
	this.designation = designation;
	this.salary = salary;
	}

	// Method to display employee details
	public void display() {
	System.out.println("ID: " + id + ", Name: " + name + ", Designation: " + designation + ",
Salary: $" + salary);
	} }

// Main class
public class EmployeeArray {
	public static void main(String[] args) {
	Scanner scanner = new Scanner(System.in);

	// Asking the user for the number of employees
	System.out.print("Enter the number of employees: ");
```

```
int numEmployees = scanner.nextInt();
scanner.nextLine(); // Consume newline

// Creating an array of Employee objects
Employee[] employees = new Employee[numEmployees];

// Accepting user input for each employee
for (int i = 0; i < numEmployees; i++) {
System.out.println("\nEnter details for Employee " + (i + 1) + ":");

System.out.print("Enter Employee ID: ");
int id = scanner.nextInt();
scanner.nextLine(); // Consume newline

System.out.print("Enter Employee Name: ");
String name = scanner.nextLine();

System.out.print("Enter Employee Designation: ");
String designation = scanner.nextLine();

System.out.print("Enter Employee Salary: ");
double salary = scanner.nextDouble();
scanner.nextLine(); // Consume newline

// Initializing the employee object
employees[i] = new Employee(id, name, designation, salary);
}

// Displaying all employee details
System.out.println("\nEmployee Details:");
for (Employee emp : employees) {
emp.display();
}

scanner.close(); // Closing the scanner
} }
```

Input:

Enter the number of employees: 3

Enter details for Employee 1:
Enter Employee ID: 101
Enter Employee Name: Alice Johnson
Enter Employee Designation: Software Engineer
Enter Employee Salary: 75000

Enter details for Employee 2:
Enter Employee ID: 102
Enter Employee Name: Bob Smith
Enter Employee Designation: Project Manager
Enter Employee Salary: 90000

Enter details for Employee 3:
Enter Employee ID: 103
Enter Employee Name: Charlie Davis
Enter Employee Designation: HR Manager
Enter Employee Salary: 65000

Output:

Employee Details:
ID: 101, Name: Alice Johnson, Designation: Software Engineer, Salary: $75000.0
ID: 102, Name: Bob Smith, Designation: Project Manager, Salary: $90000.0
ID: 103, Name: Charlie Davis, Designation: HR Manager, Salary: $65000.0

Example & Application:

This program is useful in organizations where HR teams need to store employee data dynamically and retrieve it for processingApplications

1. HR Software – Employee database management systems.
2. Payroll Management – Calculating salaries and tax deductions.
3. Company Records – Maintaining structured employee details.

Conclusion:

- This program demonstrates how to create an array of objects in Java.
- It accepts user input dynamically instead of using hardcoded values.
- It applies object-oriented programming concepts such as classes, objects, constructors, and methods.

Web/virtual links:

https://docs.oracle.com/en/java/
GeeksforGeeks - Java Arrays

References:

- *"Java: The Complete Reference"* by Herbert Schildt
- *"Head First Java"* by Kathy Sierra & Bert Bates

Experiment No: 10

Aim: Create a demo application to understand the role of access modifiers

Theory:
Access modifiers in Java determine the visibility and accessibility of classes, methods, and variables. The four main access modifiers are:

1. private: Accessible only within the same class.
2. default (package-private): Accessible within the same package.
3. protected: Accessible within the same package and subclasses.
4. public: Accessible from anywhere.

Using access modifiers effectively ensures data encapsulation, security, and proper modularization in applications.

- The user creates a bank account and attempts to access account details using different access modifiers.
- The output displays which data members and methods are accessible from different scopes.

Coding:

```
// BankAccount.java (Encapsulation using access modifiers)
class BankAccount {
        private String accountHolderName; // Private: Accessible only in this class
        private double balance; // Private: Keeps sensitive data secure

        public BankAccount(String accountHolderName, double initialBalance) {
        this.accountHolderName = accountHolderName;
        this.balance = initialBalance;
        }

        public void deposit(double amount) { // Public: Accessible from anywhere
        if (amount > 0) {
        balance += amount;
        System.out.println("Deposit successful. New balance: " + balance);
        }
        }

        public double getBalance() { // Public getter to access private balance
        return balance;
        } }

// Main class to test access modifiers
public class AccessModifierDemo {
        public static void main(String[] args) {
```

```
BankAccount account = new BankAccount("Alice", 5000);

        // account.balance = 10000; // Error: balance is private
        // System.out.println(account.accountHolderName); // Error: Private field

        // Using public methods to access private fields
        account.deposit(2000);
        System.out.println("Current Balance: " + account.getBalance());
        } }
```

Input:
Provided in the code

Output:
Deposit successful. New balance: 7000.0
Current Balance: 7000.0

Example & Application:

- Banking Systems: Securely handling customer data and transactions.
- Medical Records Management: Restricting direct access to sensitive patient data.
- E-commerce Platforms: Controlling access to order and payment details.
- Enterprise Software: Implementing encapsulation to protect business logic.

Conclusion: Access modifiers in Java provide control over the visibility and security of class members. Proper use of these modifiers enhances encapsulation, modularity, and data protection in applications. In the banking example, private attributes ensure that sensitive information is only accessible through controlled methods, preventing unauthorized modifications.

Web/virtual links: **https://docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html**

References:

- *Effective Java by Joshua Bloch*
- *Head First Java by Kathy Sierra & Bert Bates*

Experiment No: 11

Aim: Design a vehicle class hierarchy in Java, and develop a program to demonstrate Polymorphism.

Theory:

Inheritance – Allows a child class to inherit properties and behaviors from a parent class. Polymorphism – Enables a common interface for different underlying types.

Method Overriding – A subclass provides a specific implementation of a method from the parent class.

Dynamic Method Dispatch (Runtime Polymorphism) – A superclass reference can point to subclass objects and invoke overridden methods at runtime.

We create a base class Vehicle and multiple subclasses (Car, Bike, Truck) that override a method to demonstrate polymorphism. The program allows the user to choose a vehicle type at runtime.

The user selects a vehicle type by entering a number (1, 2, or 3).

A Vehicle reference (myVehicle) is used to store an object of Car, Bike, or Truck dynamically. Method Overriding ensures that the correct start() method is called based on the actual object type. Dynamic Method Dispatch allows the method call to be resolved at runtime.

Coding:

```
import java.util.Scanner;
// Base Class
class Vehicle {
void start() {
 System.out.println("The vehicle is starting.");
} }
// Subclass 1 - Car
class Car extends Vehicle {
@Override
void start() {
 System.out.println("The car starts with a key or push button.");
} }
// Subclass 2 - Bike
class Bike extends Vehicle {
@Override
void start() {
 System.out.println("The bike starts with a self-start or kick.");
} }
```

```
// Subclass 3 - Truck
class Truck extends Vehicle {
@Override
void start() {
 System.out.println("The truck starts with a heavy-duty ignition system.");
} }

// Main Class
public class VehicleDemo {
public static void main(String[] args) {
 Scanner scanner = new Scanner(System.in);
 System.out.println("Choose a vehicle type to start:");
 System.out.println("1. Car");
 System.out.println("2. Bike");
 System.out.println("3. Truck");
 System.out.print("Enter your choice (1-3): ");

 int choice = scanner.nextInt();
 Vehicle myVehicle; // Superclass reference
 switch (choice) {
 case 1:
 myVehicle = new Car();
 break;
 case 2:
 myVehicle = new Bike();
 break;
 case 3:
 myVehicle = new Truck();
 break;
 default:
 System.out.println("Invalid choice! Defaulting to a generic vehicle.");
 myVehicle = new Vehicle();
 break;
 }
 myVehicle.start(); // Calls overridden method based on object type
 scanner.close();
} }
```

Input:

```
Choose a vehicle type to start:
1. Car
2. Bike
3. Truck
Enter your choice (1-3): 2
```

Output:
The bike starts with a self-start or kick.

Example & Application:
Automobile Systems – Different vehicle types have different starting mechanisms.
Ride-Sharing Apps – The user selects a vehicle, and the app assigns the correct type dynamically.
Game Development – Vehicles in racing games have distinct behaviors.
AI Traffic Systems – Identifying vehicle types dynamically for automated controls.

Conclusion:
This program successfully demonstrates polymorphism with user input in Java. It allows dynamic selection of a vehicle type, ensuring that the correct method is executed at runtime using method overriding and dynamic method dispatch.
Web/virtual links: https://docs.oracle.com/en/java/

References:
● Java: The Complete Reference by Herbert Schildt
● Effective Java by Joshua Bloch

Experiment No: 12

Aim: Implement wait,notify,and notifyAll methods.

Theory:

- **Multithreading**: In Java, multiple threads can execute concurrently. Synchronization ensures data consistency when multiple threads access shared resources.
- **wait()**: Causes the current thread to wait until another thread invokes notify() or notifyAll() on the same object.
- **notify()**: Wakes up a single waiting thread on the same object.
- **notifyAll()**: Wakes up all waiting threads on the same object.
- **Synchronization**: Required because wait(), notify(), and notifyAll() must be called within a synchronized block or method to avoid thread interference.

This mechanism is commonly used in producer-consumer problems, where one thread produces data and another consumes it.

Coding:

```
class Message {
    private String content;
    private boolean available = false;

    public synchronized void produce(String msg) {
        while (available) { // Wait if message is not yet consumed
            try {
                wait();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        content = msg;
        available = true;
        System.out.println("Produced: " + msg);
        notify(); // Notify one consumer
    }

    public synchronized void consume() {
        while (!available) { // Wait if no message is available
            try {
                wait();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        System.out.println("Consumed: " + content);
        available = false;
```

```
        notifyAll(); // Notify all producers/consumers
    }
}

class Producer extends Thread {
    private Message msg;
    Producer(Message m) {
        msg = m;
    }
    public void run() {
        for (int i = 1; i <= 5; i++) {
            msg.produce("Message " + i);
        }
    }
}

class Consumer extends Thread {
    private Message msg;
    Consumer(Message m) {
        msg = m;
    }
    public void run() {
        for (int i = 1; i <= 5; i++) {
            msg.consume();
        }
    }
}

public class WaitNotifyDemo {
    public static void main(String[] args) {
        Message m = new Message();
        Producer p = new Producer(m);
        Consumer c = new Consumer(m);

        p.start();
        c.start();
    }
}
```

Input:
Provided in the code

Output:
Produced: Message 1
Consumed: Message 1
Produced: Message 2
Consumed: Message 2
Produced: Message 3
Consumed: Message 3
Produced: Message 4
Consumed: Message 4
Produced: Message 5
Consumed: Message 5

Example & Application:

- **Producer-Consumer problem** in operating systems.
- **Thread coordination** in parallel applications.
- **Resource management** in databases or servers.
- **Message passing** in distributed systems.

Conclusion: This experiment demonstrates how wait(), notify(), and notifyAll() are used for thread synchronization in Java. These methods help multiple threads communicate efficiently while avoiding race conditions.

Web/virtual links:
http://docs.oracle.com/javase/tutorial/essential/concurrency/guardmeth.html?utm_source=chatgpt.com

References:

- Herbert Schildt, *Java: The Complete Reference*
- Joshua Bloch, *Effective Java*

Experiment No: 13

Aim: Create and demonstrate **user-defined checked and unchecked exceptions** in Java.

Theory:

- **Exceptions** in Java represent runtime errors or unusual conditions.
- **Checked Exceptions**:
- Checked at **compile time**.
- Must be declared in the method signature using throws.
- Example: IOException, SQLException.
- **Unchecked Exceptions**:
- Checked at **runtime**.
- Inherit from RuntimeException.
- Example: NullPointerException, ArrayIndexOutOfBoundsException.
- **User-defined Exceptions**:
- Developers can create their own exception classes by extending Exception (for checked) or RuntimeException (for unchecked).
- Useful for handling application-specific errors.

Coding:

```
// User-defined Checked Exception
class InvalidAgeException extends Exception {
   public InvalidAgeException(String message) {
      super(message);
   }
}

// User-defined Unchecked Exception
class NegativeNumberException extends RuntimeException {
   public NegativeNumberException(String message) {
      super(message);
   }
}

public class CustomExceptionDemo {

   // Method that throws Checked Exception
   public static void checkAge(int age) throws InvalidAgeException {
      if (age < 18) {
         throw new InvalidAgeException("Age must be 18 or above!");
      } else {
         System.out.println("Valid Age: " + age);
      }
   }
```

```
 // Method that throws Unchecked Exception
   public static void checkNumber(int num) {
      if (num < 0) {
         throw new NegativeNumberException("Negative numbers are not allowed!");
      } else {
         System.out.println("Valid Number: " + num);
      }
   }

   public static void main(String[] args) {
      try {
         checkAge(15);  // This will throw checked exception
      } catch (InvalidAgeException e) {
         System.out.println("Caught Checked Exception: " + e.getMessage());
      }

      try {
         checkNumber(-10); // This will throw unchecked exception
      } catch (NegativeNumberException e) {
         System.out.println("Caught Unchecked Exception: " + e.getMessage());
      }

      System.out.println("Program continues...");
   }
}
```

Input:
Provided in the code

Output:

Caught Checked Exception: Age must be 18 or above!
Caught Unchecked Exception: Negative numbers are not allowed!
Program continues....

Example & Application:

- **Banking Systems** – Throwing exceptions when account balance is insufficient.
- **Student Management Systems** – Throwing exceptions for invalid age or ID.
- **E-commerce Applications** – Handling negative quantity or invalid input.

Conclusion:

This experiment successfully demonstrates the creation of **user-defined checked and unchecked exceptions** in Java. It highlights how developers can enforce business logic and handle application-specific errors gracefully.

Web/virtual links:
https://docs.oracle.com/javase/tutorial/essential/exceptions/?utm_source=chatgpt.com

References:

- **Herbert Schildt, *Java:*** *The Complete Reference*
- **Kathy Sierra & Bert Bates, *Head First Java***

Experiment No: 14

Aim: Create multiple threads using Thread class and Runnable interfaces.

Theory:

- **Multithreading**: Java allows concurrent execution of two or more parts of a program (threads) for maximum CPU utilization.
- **Thread Class**:
  - We can create a thread by extending the Thread class and overriding its run() method.
  - Start the thread using start() method.
- **Runnable Interface**:
  - Another way is to implement the Runnable interface and pass the object to a Thread constructor.
  - This approach is preferred when a class needs to extend another class.
- **Key Methods**:
  - start() → Starts the execution of a thread.
  - run() → Defines the task performed by the thread.
  - sleep(ms) → Pauses thread execution.
  - join() → Waits for a thread to finish.

Coding:

```
// Creating Thread using Thread class
class ThreadExample extends Thread {
   private String name;

   ThreadExample(String name) {
      this.name = name;
   }

   public void run() {
      for (int i = 1; i <= 5; i++) {
         System.out.println(name + " (Thread Class) - Count: " + i);
         try {
            Thread.sleep(500); // Pause for 0.5 sec
         } catch (InterruptedException e) {
            System.out.println(e);
         }
      }
   }
}
```

```
// Creating Thread using Runnable interface
class RunnableExample implements Runnable {
    private String name;

    RunnableExample(String name) {
        this.name = name;
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(name + " (Runnable) - Count: " + i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

public class MultiThreadDemo {
    public static void main(String[] args) {
        // Thread using Thread class
        ThreadExample t1 = new ThreadExample("Thread-1");
        ThreadExample t2 = new ThreadExample("Thread-2");

        // Thread using Runnable interface
        RunnableExample r1 = new RunnableExample("Runnable-1");
        RunnableExample r2 = new RunnableExample("Runnable-2");

        Thread tr1 = new Thread(r1);
        Thread tr2 = new Thread(r2);

        // Start all threads
        t1.start();
        t2.start();
        tr1.start();
        tr2.start();
    }
}
```

Input:
Provided in the code

Output:

Thread-1 (Thread Class) - Count: 1
Runnable-1 (Runnable) - Count: 1
Thread-2 (Thread Class) - Count: 1
Runnable-2 (Runnable) - Count: 1
Thread-1 (Thread Class) - Count: 2
Runnable-1 (Runnable) - Count: 2
...

Example & Application:

- **Web servers** – Handling multiple client requests.
- **Gaming applications** – Running background music, user input, and rendering simultaneously.
- **Banking systems** – Processing multiple transactions at the same time.
- **Operating systems** – Managing processes in parallel.

Conclusion:

This experiment successfully demonstrates the creation of multiple threads in Java using both the **Thread class** and **Runnable interface**. It shows how multithreading allows concurrent task execution.

Web/virtual links:

https://docs.oracle.com/javase/tutorial/essential/concurrency/index.html?utm_source=chatgpt.com

References:

- **Herbert Schildt, *Java:* *The Complete Reference***
- **Kathy Sierra & Bert Bates, *Head First Java***

Experiment No: 15

Aim: Program to demonstrate implementation of AWT in Java using various elements such as Frame, Components, layout manager, event handling.

Theory:

- **AWT (Abstract Window Toolkit)**: Java's GUI (Graphical User Interface) library for creating desktop applications.
- **Frame**: A top-level window with a title, border, and buttons to close, resize, or minimize.
- **Components**: GUI elements like Button, Label, TextField, Checkbox, etc.
- **Layout Managers**: Control how components are arranged in a container. Examples: FlowLayout, BorderLayout, GridLayout.
- **Event Handling**: Mechanism by which user interactions (clicks, typing, etc.) are handled using **event listeners**.
    - **ActionListener** is commonly used to handle button click events.

Coding:

```
import java.awt.*;
import java.awt.event.*;

class AWTExample extends Frame implements ActionListener {
   Label label;
   TextField textField;
   Button button;

   AWTExample() {
      // Set frame title and size
      setTitle("AWT Example");
      setSize(400, 200);
      setLayout(new FlowLayout());

      // Create components
      label = new Label("Enter your name: ");
      textField = new TextField(20);
      button = new Button("Submit");

      // Add components to frame
      add(label);
      add(textField);
      add(button);

      // Add event listener
      button.addActionListener(this);
```

```
        // Closing the window
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                dispose();
            }
        });

        setVisible(true);
    }

    // Event handling method
    public void actionPerformed(ActionEvent e) {
        String name = textField.getText();
        label.setText("Hello, " + name + "!");
    }

    public static void main(String[] args) {
        new AWTExample();
    }
}
```

Input:
Provided in the code

Output:

1. A window (Frame) appears with a label, text field, and button.
2. User types a name in the text field.
3. On clicking **Submit**, the label changes to:

Hello, <Name>!

Example & Application:

- **Form-based applications** – Registration/Login forms.
- **Desktop utilities** – Calculators, text editors.
- **Interactive GUIs** – Games or educational apps.
- **Prototyping tools** – For GUI design

Conclusion:

This experiment demonstrates how to build a simple GUI application using **Java AWT**. It covers **Frames, Components, Layout Managers, and Event Handling**, which are essential for interactive desktop applications.

Web/virtual links:
https://docs.oracle.com/javase/8/docs/api/java/awt/package-summary.html?utm_source=chatgpt.com

References:

- **Herbert Schildt,** ***Java: The Complete Reference***
- **Patrick Naughton & Herbert Schildt,** ***Java 2: The Complete Reference***